

Multimodal Classification: Dimensionality Reduction and Unsupervised Clustering

Nikos Evangelou

Department of Computer Science, Aristotle University of Thessaloniki, Greece

nevangelou@csd.auth.gr

1. Introduction

This report details the implementation, execution, and evaluation of foundational pattern recognition algorithms for dimensionality reduction, unsupervised clustering, and supervised classification on the MNIST dataset.

Before proceeding with any implementation, the data was loaded and subjected to a preprocessing phase. The data was converted to float32 format and normalized to the $[0, 1]$ range by dividing by 255. Furthermore, to eliminate translation variance, spatial image centering is applied, aligning each digit's center of mass with the geometric center of the canvas. As established in our previous evaluation of linear models [1], this spatial transformation significantly improves baseline algorithmic performance.

2. Part A: Dimensionality Reduction - Principal Component Analysis (PCA)

2.1. Custom PCA Implementation

```
class CustomPCA:
    def __init__(self, n_components):
        # The number of components to keep
        self.n_components = n_components
        self.mean = None
        self.components = None
        self.explained_variance_ratio = None

    def fit(self, X):
        self.mean = np.mean(X, axis=0)
        X_centered = X - self.mean

        covariance_matrix = (X_centered.T @ X_centered) / (X_centered.shape[0] - 1)

        eigenvalues, eigenvectors = np.linalg.eigh(covariance_matrix)

        indices = np.argsort(eigenvalues)[::-1]
        eigenvalues_sorted = eigenvalues[indices]
        eigenvectors_sorted = eigenvectors[:, indices]

        self.components = eigenvectors_sorted[:, :self.n_components]

        # Calculate the explained variance ratio for the selected components
        total_variance = np.sum(eigenvalues_sorted)
        self.explained_variance_ratio = eigenvalues_sorted[:self.n_components] / total_variance

    def transform(self, X):
        X_centered = X - self.mean
        X_pca = np.dot(X_centered, self.components)
        return X_pca
```

Figure 1. Custom PCA Implementation

Let the dataset be X with dimensions $N \times D$, where N is the number of samples and D is the number of features (784 for the MNIST dataset).

We compute the mean vector of each column (each feature), resulting in a $1 \times D$ vector μ :

$$\mu_j = \frac{1}{N} \sum_{i=1}^N X_{i,j}$$

We subtract μ from the original matrix X to compute the centered data matrix X_c :

$$X_c = X - \mu$$

We compute the covariance matrix Σ , which describes how each pixel changes in relation to the rest, by multiplying the transpose of the centered data matrix by the centered data matrix itself, and dividing by $N - 1$. We divide by $N - 1$ rather than N to apply Bessel's correction, accounting for the fact that we are analyzing a sample rather than the entire population:

$$\Sigma = \frac{1}{N - 1} X_c^T X_c$$

We compute the eigenvalues and eigenvectors of the covariance matrix. Each eigenvalue represents the variance captured along its corresponding direction, and each eigenvector represents a principal component.

We sort the eigenvalues in descending order and sort their corresponding eigenvectors accordingly.

Finally, we keep only the first $n_components$ columns of the sorted eigenvector matrix to form the projection matrix.

2.2. Subset Creation: Classes 0, 1, and 8

The selection of the three classes was not arbitrary. We selected the digits 0, 1, and 8 due to their morphological diversity:

- **Digit 1:** Represents the simplest structure.
- **Digit 0:** Represents a different simple morphology.
- **Digit 8:** Represents a more complex shape.

This selection helps the K-Means algorithm (in Part B) create distinct clustering regions, for the sake of visualization. This structural diversity prevents the overlap of the classes, ensuring that the Voronoi diagrams clearly shows the predicted clusters.

2.3. Cumulative Variance Ratio & Component Selection

Figure 2 illustrates the cumulative explained variance ratio across the first 10 principal components. As observed, the first two principal components capture approximately 34% of the total variance. The inclusion of a third component increases the captured variance to 42%. While the

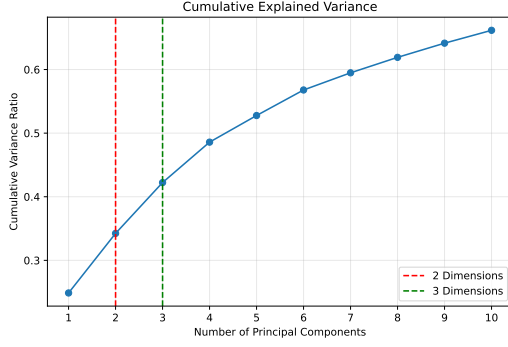


Figure 2. Cumulative explained variance ratio across the first 10 principal components.

3-dimensional projection retains a higher portion of the dataset’s variance, we opted for a 2-dimensional PCA projection. We chose the 2D projection because it makes the separated clusters easier to see. It also solves the problem of points hiding behind one another, which often happens in 3D graphs.

2.4. 2D Visualization

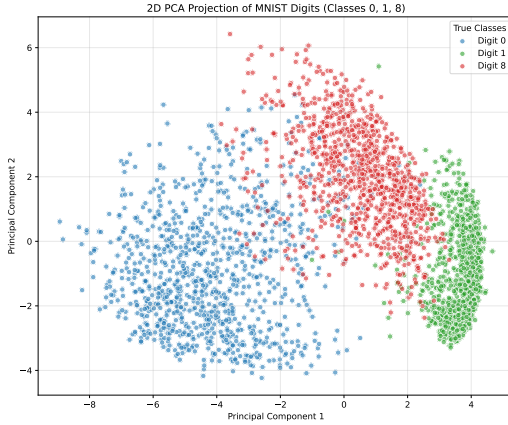


Figure 3. 2D PCA projection of the MNIST subset (Classes 0, 1, and 8).

Figure 3 presents the 2D PCA projection of the MNIST subset comprising classes 0, 1, and 8. The visualization confirms that although PCA is an unsupervised method (it never accessed the labels during training), and the reduction in dimensions was massive (from 784 to 2 dimensions), it successfully captures the structural patterns of the digits, forming three distinct spatial regions.

Specifically:

- **Digit 1 (Green)** forms an extremely dense and compact cluster with minimal intra-class variance, isolating itself. This behavior is explained by its simple, strictly linear morphology, which leaves little room for structural variation.
- **Digit 0 (Blue) and Digit 8 (Red)** exhibit significantly higher intra-class variance, forming wider

spatial distributions. Despite their topological similarities (both containing closed loops), the projection separates them effectively, with only marginal overlap at their adjoining boundaries.

2.5. Eigen-digits and Pixel Saliency

To interpret the spatial transformations applied by the PCA, we visualize the learned weights in the original 28×28 image space.

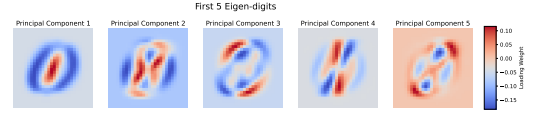


Figure 4. Visualization of the first five principal components as Eigen-digits.

Figure 4 visualizes the first five principal components as “Eigen-digits”, utilizing a diverging colormap to represent the loading weights.

- **Principal Component 1** acts as a primary structural discriminator. It exhibits strong positive weights (red) in the central vertical axis and strong negative weights (blue) in the outer circular regions. This demonstrates that PC1 captures the contrast between the narrow geometry of the digit ‘1’ and the wide, looped geometries of ‘0’ and ‘8’.
- **Principal Components 2 through 5** capture increasingly complex spatial variations. PC2, for instance, highlights the internal crossbar of the digit ‘8’ versus the empty center of the digit ‘0’.

These visualizations validate that the PCA projection is a geometric encoding of the distinct morphological features present in the selected classes.

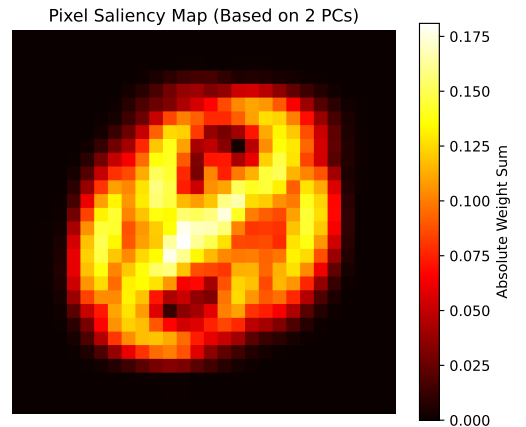


Figure 5. Pixel Saliency Map calculated from the first two principal components.

Figure 5 displays the Pixel Saliency Map, calculated as the absolute weight sum of the first two principal components. The map clearly identifies the informative regions

of the dataset. The dark background indicates near-zero weight, confirming that PCA correctly ignores the zero-variance background pixels around the digits. The highest values form a bright vertical core and a distinct outer ring.

3. Part B: Unsupervised Clustering (k-Means) in Low-Dimensional Space

3.1. Custom k-Means Implementation

```
class CustomKMeansPlusPlus:
    def __init__(self, k, max_iter=1000, tol=1e-5):
        self.k = k
        self.cluster_centroids = None
        self.max_iter = max_iter
        self.tol = tol

    def _compute_distances(self, X, centroids):
        # Calculate pairwise Euclidean distances using the algebraic expansion:
        # ||x - c||^2 = ||x||^2 + ||c||^2 - 2x·c
        x_square = np.sum(X**2, axis=1, keepdims=True)
        centroids_square = np.sum(centroids**2, axis=1, keepdims=True).T
        dot_product = np.dot(X, centroids.T)
        distances = x_square + centroids_square - 2 * dot_product

        # Handle possible negative floating point numbers, caused by floating point calculation inaccuracies
        distances = np.maximum(distances, 0)

        return np.sqrt(distances)

    def fit(self, X):
        self.cluster_centroids = np.zeros((self.k, X.shape[1]))
        self.cluster_centroids[0] = X[np.random.choice(X.shape[0])]

        # Initialization (K-Means++)
        for i in range(1, self.k):
            distances = self._compute_distances(X, self.cluster_centroids[:i])
            min_distances = np.min(distances, axis=1)
            probabilities = min_distances**2 / np.sum(min_distances**2)

            self.cluster_centroids[i] = X[np.random.choice(X.shape[0], p=probabilities)]

        # Assign each sample to the nearest centroid
        for _ in range(self.max_iter):
            distances = self._compute_distances(X, self.cluster_centroids)
            labels = np.argmax(distances, axis=1)

            old_centroids = self.cluster_centroids.copy()
            for i in range(self.k):
                cluster_i_samples = X[labels == i]
                if len(cluster_i_samples) > 0:
                    self.cluster_centroids[i] = np.mean(cluster_i_samples, axis=0)
                # If a cluster is empty, move its centroid to the sample that is the furthest from each centroid
                else:
                    furthest_idx = np.argmax(np.min(distances, axis=1))
                    self.cluster_centroids[i] = X[furthest_idx]

            if np.linalg.norm(self.cluster_centroids - old_centroids) < self.tol:
                break

    def predict(self, X):
        distances = self._compute_distances(X, self.cluster_centroids)
        return np.argmax(distances, axis=1)
```

Figure 6. Custom K-Means++ Initialization.

To overcome the instability of completely random centroids initialization, we implement the K-Means++ strategy. This ensures that the initial cluster centroids are placed as far away from each other as possible. Figure 6 shows the full code implementation for K-Means algorithm.

The algorithm consists of two main phases: Initialization (K-Means++) and the standard optimization loop.

Phase 1: K-Means++ Initialization

- 1) Select the first cluster centroid randomly from the dataset samples.
- 2) Repeat the following steps until we have gathered K cluster centroids:
 - For each sample x_i , we compute its minimum Euclidean distance from the nearest already chosen centroid $D(x_i)$:

$$D(x_i) = \min_{c \in C_{\text{chosen}}} \|x_i - c\|_2$$

- We compute the probability $p(x_i)$ for each sample to be selected as the next centroid, which is proportional to its squared distance:

$$p(x_i) = \frac{D(x_i)^2}{\sum_{j=1}^N D(x_j)^2}$$

- We select the next centroid from the dataset samples according to this probability distribution.

Phase 2: K-Means Optimization Once the K initial centroids are established, the standard algorithm iteratively optimizes their spatial positions:

- 1) **Assignment Step:** Assign each data point x_i to the nearest cluster centroid based on the minimum Euclidean distance.
- 2) **Update Step:** Recalculate each centroid as the mean vector of all data points currently assigned to its cluster.
- 3) **Convergence:** Repeat the Assignment and Update steps until the cluster assignments no longer change more than a predefined threshold.

3.2. K-Means Clustering on 2D Projections & Performance Evaluation

Table 1 details the exact clustering assignments. Digit 1 forms the purest cluster, with only 22 samples grouped falsely, while digits 0 and 8 exhibit higher confusion, as 139 zeros and 158 eights are misassigned to incorrect clusters due to their morphological similarities.

To further evaluate the K-Means clustering, we used two complementary metrics: the Silhouette Score, and the Adjusted Rand Index (ARI).

- **Silhouette Score (0.5279):** This unsupervised metric measures cluster cohesion and separation on a scale from -1 to 1. Our score of 0.5279 is a solid result, indicating that the algorithm successfully formed distinct clusters. However, the deviation from 1.0 reflects a degree of geometric overlap at the cluster borders, meaning some data points are too close to neighboring clusters.
- **Adjusted Rand Index - ARI (0.7004):** ARI measures the agreement between the predicted clusters and the true labels (scale from -1 to 1). A score of 0.7004 is a strong result. It suggests that despite operating without labels, K-Means successfully identified and separated the digit classes in the 2D space. The remaining error indicates that misclassifications naturally occur at the boundary regions where clusters overlap.

3.3. Comparative Visualization and Voronoi Boundaries

To visually evaluate the clustering performance, Figure 7 provides a single combined plot. The data points are

TABLE 1. CONFUSION MATRIX: TRUE LABELS VS K-MEANS ASSIGNMENTS.

Cluster True Label	0	1	2
0	855	6	139
1	0	978	22
8	14	158	828

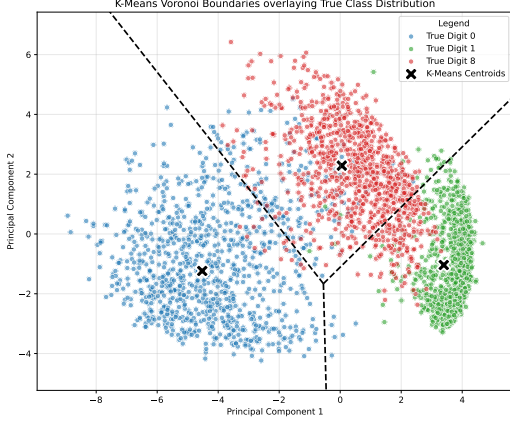


Figure 7. Comparative visualization of true labels and predicted K-Means clusters overlaid with Voronoi boundaries.

colored according to their true labels, while the dashed lines represent the Voronoi boundaries generated by the K-Means algorithm. The black "X" markers indicate the final cluster centroids.

This visualization clearly demonstrates why the ARI score is approximately 0.70. The K-Means algorithm successfully isolated digit 1 (green), drawing a clean boundary that captures almost all of its instances. However, because digits 0 (blue) and 8 (red) are more similar, their spatial distributions overlap in the center. Since K-Means enforces strict linear boundaries, we can visibly see several points belonging to digit 0 falling into the Voronoi cell of digit 8, and vice versa. This confirms that while the algorithm captures the general structure of the data, linear separation is insufficient for perfectly distinguishing overlapping classes.

3.4. Centroid Reconstruction in Original High-Dimensional Space

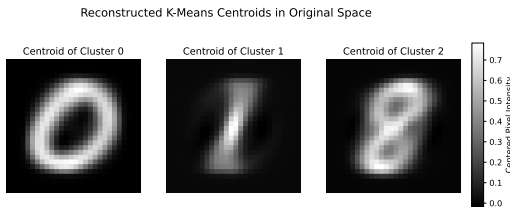


Figure 8. Reconstruction of the 2D K-Means cluster centroids back into the original 784-dimensional image space.

To understand what the K-Means algorithm learned, Figure 8 illustrates the projection of the 2D cluster centroids back into the original 784-dimensional pixel space. This inverse transformation was performed by multiplying the 2D centroids by the transposed PCA components matrix and adding the dataset's mean vector. This process restores the cluster centroids to their original feature space, allowing us to reshape the resulting 784-dimensional vectors into 28×28 images.

The visual results confirm that the algorithm successfully captured the geometric archetypes of the target digits:

- **Cluster 0:** Displays a distinct circular outer ring, characteristic of the digit 0.
- **Cluster 1:** Features a centered vertical stroke, representing the digit 1.
- **Cluster 2:** Shows the dual-loop structure forming the shape of the digit 8, while also exhibiting a little low-intensity outer circle, reflecting the overlap with instances of the digit 0.

This visualization demonstrates that the mathematical centers identified by the unsupervised K-Means algorithm correspond nicely to the structural patterns of the handwritten digits, confirming the effectiveness of the PCA projection followed by clustering.

4. Part C: Multimodal Classification via Mahalanobis Distance

For the remainder of this study, the full dataset comprising all 10 classes is utilized, rather than the restricted 3-class subset.

4.1. PCA as a Prerequisite for Mahalanobis Distance

Applying Principal Component Analysis (PCA) prior to computing the Mahalanobis distance is a mathematical necessity for datasets like MNIST. The calculation of the Mahalanobis distance relies on the inverse of the class covariance matrix (Σ^{-1}). In the original 784-dimensional space, many background pixels are the same across all samples, resulting in zero variance. This renders the covariance matrix non-invertible, making the distance calculation impossible. By applying PCA and projecting the data into a lower-dimensional subspace, we eliminate these zero-variance features, resolving the problem.

4.2. Euclidean-based Nearest Centroid Classifier

Suppose we use the simple scikit-learn Euclidean-based Nearest Centroid Classifier (NCC) to predict the digit labels. The accuracy achieved on the 10D PCA-projected data is 0.7614.

The standard NCC assumes spherical covariance (utilizing the Euclidean distance). We proceed to a more sophisticated approach, allowing each class to have its own ellipsoidal shape, which geometrically translates to computing a distinct covariance matrix (Σ_c) for each class.

4.3. Custom Mahalanobis-based Nearest Centroid Classifier Implementation

```
class CustomMahalanobisNCC:
    def __init__(self):
        self.class_mean = {}
        self.class_inv_cov = {}
        self.subclass_mapping = None

    def fit(self, X, y, subclass_mapping=None):
        self.subclass_mapping = subclass_mapping
        classes = np.unique(y)
        for c in classes:
            class_indices = np.where(y == c)[0]
            class_data = X[class_indices]

            self.class_mean[c] = np.mean(class_data, axis=0)

            class_cov = np.cov(class_data, rowvar=False)
            class_cov += 1e-5 * np.eye(class_cov.shape[0])
            self.class_inv_cov[c] = np.linalg.inv(class_cov)

    def predict(self, X):
        classes = list(self.class_mean.keys())
        distances_sq = np.zeros((X.shape[0], len(classes)))

        for i, c in enumerate(classes):
            mean_diff = X - self.class_mean[c]
            distances_sq[:, i] = np.sum(np.dot(mean_diff, self.class_inv_cov[c]) * mean_diff, axis=1)

        min_indices = np.argmin(distances_sq, axis=1)
        predictions = np.array([classes[i] for i in min_indices])

        if self.subclass_mapping is not None:
            predictions = np.array([self.subclass_mapping[pred] for pred in predictions])

        return predictions
```

Figure 9. Implementation structure of the Custom Mahalanobis NCC.

As observed in Figure 9 during the training phase, for each class c , we compute:

- 1) The mean vector (centroid) of class c :

$$\mu_c = \frac{1}{N_c} \sum_{i=1}^{N_c} x_i$$

Where N_c is the number of samples of class c and x_i represents each sample's feature vector.

- 2) The inverse covariance matrix Σ_c^{-1} , which is computed by inverting the class covariance matrix:

$$\Sigma_c = \frac{1}{N_c} \sum_{i=1}^{N_c} (x_i - \mu_c)(x_i - \mu_c)^T$$

To prevent numerical instability caused by floating-point inaccuracies, a small regularization term is added to the diagonal:

$$\Sigma_c \leftarrow \Sigma_c + \epsilon I$$

Where ϵ is a sufficiently small constant (e.g., 10^{-5}) and I is the identity matrix.

During the inference phase, for each test sample x , we compute the squared Mahalanobis distance to each class centroid μ_c :

$$D_M^2(x, \mu_c) = (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c)$$

The sample x is then assigned to the predicted class $\hat{y}$ that yields the minimum distance:

$$\hat{y} = \arg \min_c D_M^2(x, \mu_c)$$

4.3.1. Relationship between Mahalanobis Distance and Maximum Likelihood Estimation. The mathematical rationale behind this approach is closely related to the principles of Maximum Likelihood for Multivariate Gaussian distributions.

The MLE method seeks to identify the parameters that maximize the likelihood of the observed training data. For a Multivariate Gaussian distribution, the solutions that maximize this likelihood are the sample mean vector (μ_c) and the sample covariance matrix (Σ_c). These are the exact same parameters computed during the training phase of our custom Mahalanobis-based Nearest Centroid Classifier.

During the inference phase, assuming a Multivariate Gaussian distribution, the log-likelihood of a new test sample x belonging to class c is proportional to:

$$\log P(x|c) \propto -\frac{1}{2} \log |\Sigma_c| - \frac{1}{2} (x - \mu_c)^T \Sigma_c^{-1} (x - \mu_c)$$

In our custom Mahalanobis-based Nearest Centroid Classifier, classification is performed by minimizing the squared Mahalanobis distance. As observed, this squared distance is identical to the second term of the $\log P(x|c)$ equation (excluding the $-\frac{1}{2}$ constant).

Therefore, the Mahalanobis distance corresponds to the quadratic term of the log-likelihood for a (Multivariate) Gaussian distribution. Consequently, our classifier maximizes the distance-based component of the likelihood, functioning as a Maximum Likelihood Classifier only under the assumption that all classes have equal covariance determinants ($|\Sigma_c| = |\Sigma|$), since the log-based term then becomes constant across all classes and the prediction becomes fully driven by the Mahalanobis-based distance term.

4.4. Mahalanobis-based Classification

The Mahalanobis distance metric significantly improves classification performance, yielding an accuracy of 0.8929 for the 10 original classes.

As shown in Figure 10, the model performs well but struggles in some classes. The highest misclassification rates were observed for digits '4' and '7'.

The standard Nearest Centroid Classifier assumes one centroid per class (one Gaussian distribution for each digit). Handwritten digits can be written in many ways, exhibiting multi-modality. For example, the digit '7' can be written with or without a central horizontal line, which explains why it was misclassified so much.

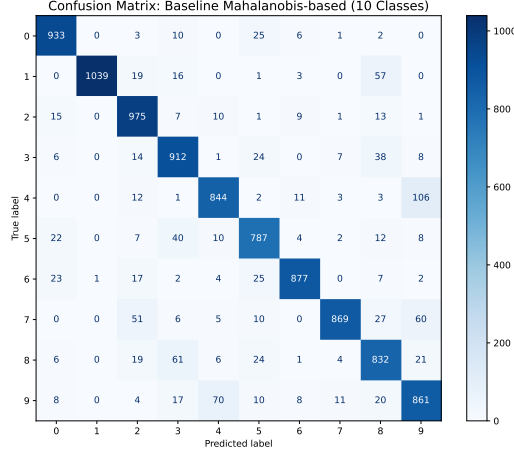


Figure 10. Confusion Matrix for the 10-class Mahalanobis classification.

4.5. Mahalanobis-based Classification with 2 Subclasses per digit

To address the issue mentioned above, we introduce multi-modality into the model by utilizing K-Means to partition each class into 2 distinct subclasses. By splitting each class into multiple sub-distributions, the model adapts to additional writing styles, capturing these intra-class variations. Expanding the feature space representation to 20 subclasses (2 clusters per digit) results in an improved accuracy of 0.9022.

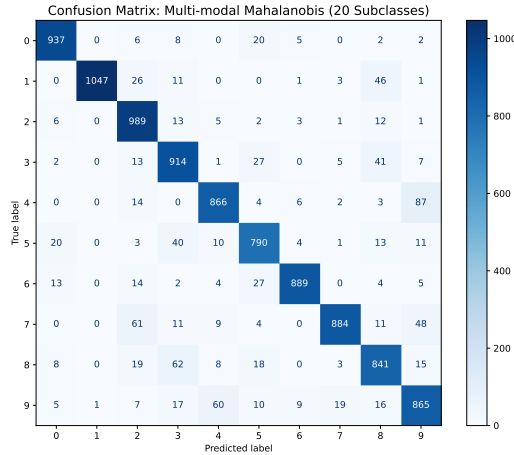


Figure 11. Confusion Matrix for the multimodal Mahalanobis classification utilizing 20 subclasses.

Figure 11 confirms that multi-modality effectively reduces the model’s confusion errors. Specifically, the correct classification of 4 and 7 increased by 22 and 15 respectively, demonstrating that the model better differentiates between distinct morphological variations of the same digit. This refinement validates that capturing intra-class diversity is crucial for improving recognition robustness.

5. Further Experiment: L2-normalized PCA and Cosine Equivalence

Each MNIST image (784 pixels) is represented as a vector x in the space $\mathbb{R}^{784}$. The length (L_2 norm) of this vector is calculated as:

$$\|x\|_2 = \sqrt{\sum_{i=1}^{784} x_i^2}$$

In the context of MNIST (where values are pixel intensities), the length $\|x\|_2$ practically expresses how much ink was used for the digit. Meanwhile, the angle of the vector expresses the structure of the digit.

5.1. The L_2 -Normalization Process:

We divide each vector by its own length, creating a new vector $\hat{x}$:

$$\hat{x} = \frac{x}{\|x\|_2}$$

Now, it strictly holds that $\|\hat{x}\|_2 = 1$ for all samples. Each vector now has the same length ($\|\hat{x}\|_2 = 1$) but a different angle. Therefore, each digit is projected onto the surface of a hypersphere of radius 1. Variations related to the overall magnitude of the pixel values are removed. This means that comparisons between digits are now based primarily on shape-related characteristics of the digit rather than overall image intensity. In other words, the distance is now determined by directional differences between the vectors, which are mostly related to the structure of the digit.

5.2. Mathematical Equivalence of Euclidean and Cosine:

The Euclidean distance between two normalized vectors $\hat{x}$ and $\hat{y}$ is:

$$D_{Euc}^2(\hat{x}, \hat{y}) = \|\hat{x} - \hat{y}\|_2^2 = \|\hat{x}\|_2^2 + \|\hat{y}\|_2^2 - 2\hat{x}^T \hat{y}$$

Since $\|\hat{x}\|_2^2 = 1$ and $\|\hat{y}\|_2^2 = 1$, and the inner product $\hat{x}^T \hat{y}$ is equal to the cosine of their angle ($\cos \theta$), we have:

$$D_{Euc}^2(\hat{x}, \hat{y}) = 2 - 2 \cos \theta$$

Conclusion: When all samples are L_2 -normalized, Euclidean distances depend exclusively on the angle between vectors, completely ignoring their length. Consequently, Euclidean distance between normalized vectors becomes a monotonic function of cosine similarity. PCA and Mahalanobis classification are still computed in the normalized feature space, but they operate on data where magnitude/length information has been removed.

In the L_2 -normalized feature space, covariance reflects variations in vector directions rather than variations in vector magnitude, since all samples are constrained to have unit norm.

TABLE 2. ABLATION STUDY DEMONSTRATING THE EFFECTS OF DISTANCE METRICS, MULTIMODAL CLUSTERING, AND GEOMETRIC TRANSFORMATIONS (SPHERICAL PCA) ON CLASSIFICATION.

Distance Metric	Total Clusters	Feature Space	Accuracy
Euclidean	1	Standard	0.7614
Euclidean	1	L2-normalized	0.7661
Euclidean	2	Standard	0.8095
Euclidean	2	L2-normalized	0.8199
Mahalanobis	1	Standard	0.8929
Mahalanobis	1	L2-normalized	0.9033
Mahalanobis	2	Standard	0.9022
Mahalanobis	2	L2-normalized	0.9132

6. Ablation Study Results

Table 2 summarizes how each method improves classification accuracy.

Impact of the Distance Metric: Switching from Euclidean to Mahalanobis distance creates the biggest improvement, jumping from 0.7614 to 0.8929. This suggests that digits have complex, ellipsoidal distributions, making the simple Euclidean distance inadequate.

Impact of Multimodal Clustering: Splitting each digit into two subclasses consistently improves results. It helps the Euclidean model significantly (+4.8%) and gives a vital boost to the Mahalanobis model (+0.93%) by adapting to different writing styles.

Impact of Spherical PCA: Applying L_2 -normalization before PCA adds about 0.5% to 1.1% to the accuracy across all tests. It reduces the influence of pixel intensity, thus focusing more on the digit’s shape.

Conclusion: The combination of the three techniques yields the highest observed accuracy. The Mahalanobis metric handles data spread, subclasses capture different writing styles, and Spherical PCA removes ink intensity noise. Combined, they achieve the highest accuracy of 0.9132.

Declarations

During the preparation of this manuscript, Large Language Models (LLMs) were utilized to assist with code generation, debugging, and language refinement. The originality of the experimental analysis remains entirely intact.

References

- [1] N. Evangelou, “Linear Classifiers: Class Imbalance and Distribution Shifts,” *Technical Report, Dept. of Computer Science, Aristotle University of Thessaloniki*, 2026.
- [2] Y. LeCun, C. Cortes, and C. J. Burges, “Mnist handwritten digit database,” *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, vol. 2, 2010.